

[← Go Back](#)

Hardware

Portenta C33



Tutorials

Energy Metering with the
Portenta C33

**Portenta C33 User
Manual**

Home / Hardware / Portenta C33 / **Portenta C33
User Manual**

Portenta C33 User Manual

Learn about the hardware and software features of the
Arduino® Portenta C33.

Author • José Bagur

Last revision • 09/02/2024

ON THIS PAGE

Overview

Hardware and Software Requirements	+
Product Overview	+
First Use	+
Pins	+
Actuators	+
Communication	+
External Memory	
Secure Element	
Arduino Cloud	
Support	+

Overview

This user manual will provide a comprehensive overview of the Portenta C33 board, covering its major hardware and software elements. With this user manual, you will learn how to set up, configure and use all the main features of the board.

Hardware and Software Requirements

Hardware Requirements

Portenta C33 (x1)

USB-C® cable (x1)

Wi-Fi® W.FL antenna (x1)

Software Requirements

Arduino IDE 1.8.10+, **Arduino IDE 2.0+**, or
Arduino Cloud Editor

Product Overview

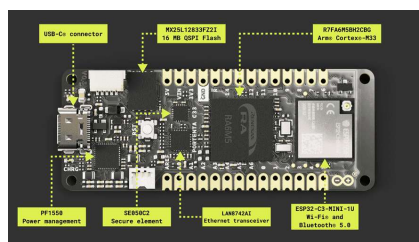
The Portenta C33 is a powerful System-on-Module (SOM) designed for cost-effective Internet of Things (IoT) applications and devices. Based on the R7FA6M5BH2CBG microcontroller from Renesas®, it shares the same form factor as the Portenta H7 board. It is backward-compatible with the MKR and Portenta family shields and carriers through its MKR-styled and High-Density

[Help](#)

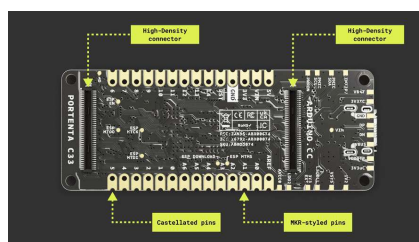
range of applications, from smart home devices to connected industrial sensors.

Board Architecture Overview

The Portenta C33 features a robust and efficient architecture that enables it for low-cost, battery-powered, IoT applications and devices.



The Portenta C33 main components
(top view)



The Portenta C33 main components
(bottom view)

Here is an overview of the board's main components shown in the images above:

Microcontroller: At the heart of the Portenta C33 is the **R7FA6M5BH2CBG**, a powerful and versatile SOM from Renesas®. The R7FA6M5BH2CBG is built around a 32-bit Arm® Cortex®-M33 processor running at 200 MHz, with 2 MB of Flash memory and 512 kB of SRAM.

External memory: The board features an onboard 16 MB QSPI Flash memory.

Wireless connectivity: The board supports 2.4 GHz Wi-Fi® (802.11 b/g/n) and Bluetooth® 5.0, provided by the ESP32-C3-MINI-1U module developed by Espressif Systems®. This high-performance Wi-Fi® and Bluetooth® module allows the Portenta C33 to communicate wirelessly with other devices and systems. Note: [ArduinoBLE](#) library support is limited to Bluetooth® 4.0 only actually.

Ethernet connectivity: The board features

Security: The board features an onboard ready-to-use secure element, the SE050C2 from NXP®, specifically designed for IoT devices and provides advanced security features.

USB connectivity: The board features a USB-C port for power and data, which is also accessible through the board's High-Density connectors.

Power management: The Portenta C33 is designed for low-power operation to meet the demands of always-connected IoT devices. It features a power management integrated circuit (PMIC), the PF1550 from NXP®, designed specifically for low-power, portable, and battery-powered IoT applications.

Analog and digital peripherals: The board features analog peripherals such as two 8-channel 12-bit analog-to-digital converters (ADC) and two 12-bit digital-to-analog converters (DAC). It also features the following digital peripherals: GPIO (x7), I2C (x1), UART (x4), SPI (x2), PWM (x10), CAN (x2), I2S (x1), SPDIF (x1), and SAI (x1).

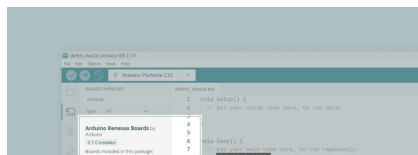
Debugging: The board features a JTAG/SWD debug port accessible through its High-Density connectors.

Surface mount: The castellated pins of the board allow it to be positioned as a surface-mountable module.

MKR-styled connectors: The MKR-styled connectors of the board make it compatible with all the MKR family boards. 2.54 mm pitch headers can be easily soldered to the board.

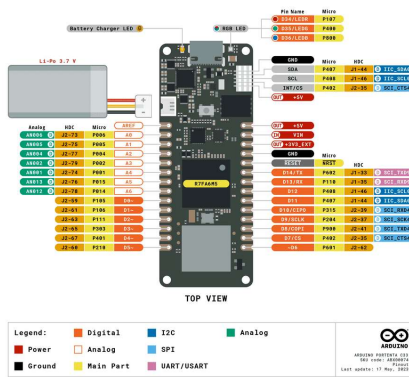
Board Core and Libraries

The **Arduino Renesas Boards** core contains the libraries and examples to work with the Portenta C33's peripherals and onboard components, such as its external QSPI Flash memory and Wi-Fi® and Bluetooth® module. To install the core for the Portenta C33 board, navigate to **Tools > Board > Boards Manager** or click the Boards Manager icon in the left tab of the IDE. In the Boards Manager tab, search for `renesas` and install the latest Arduino Renesas Boards version.



Installing the Arduino Renesas Boards core in the Arduino IDE

Pinout



Portenta C33 MKR-styled connectors pinout

The complete pinout is available and downloadable as PDF from the link below:

[Portenta C33 pinout](#)

Datasheet

The complete datasheet is available and downloadable as PDF from the link below:

[Portenta C33 datasheet](#)

Schematics

The complete schematics are available and downloadable as PDF from the link below:

[Portenta C33 schematics](#)

STEP Files

The complete STEP files are available and downloadable from the link below:

[Portenta C33 STEP files](#)

First Use

The Portenta C33 can be powered by the following:

Using a USB-C® cable (not included).

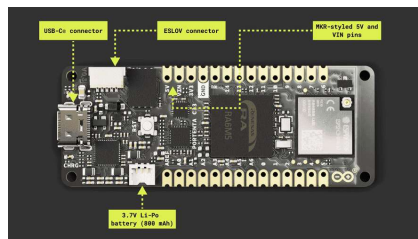
Using an external **4.1 to 6 V power supply** connected to the `VIN` pin located in the MKR-styled connectors pins of the board (please refer to the [board pinout section](#) of the user manual).

Using an external **5 V power supply** connected to the `5V` pin located in the MKR-styled connectors of the board (please refer to the [board pinout section](#) of the user manual).

Using a **3.7 V Lithium Polymer (Li-Po) battery** connected to the board through the onboard battery connector; the manufacturer part number of the battery connector is BM03B-ACHSS-GAN-TF(LF)(SN), and its matching receptacle manufacturer part number is ACHR-03V-S. The **recommended minimum battery capacity for the Portenta C33 is 800 mAh**. A Li-Po battery with an integrated NTC thermistor is also recommended for thermal protection.

Using an Arduino Pro carrier board, such as the [Portenta Max Carrier](#) and the [Portenta Breakout](#).

Using your custom boards to power the board through the board's castellated pins, MKR-styled, or High-Density connectors. Check out the [board pinout](#) and its schematic to know more about it.



Different ways to power the Portenta C33 board

Hello World Example

Let's program the Portenta C33 with the classic `hello world` example used in the Arduino ecosystem: the `Blink` sketch. We will use this example to verify the board's connection to the Arduino IDE and that the Portenta C33 core and the board itself are working as expected.

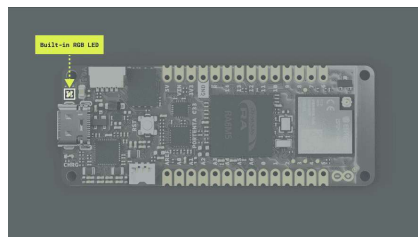
There are two ways to program this example in the board:

Navigate to **File > Examples > 01.Basics > Blink**.

Copy and paste the code below into a new sketch in the Arduino IDE.

```
1 void setup() {  
2   // Initialize LED_BUILTIN as an output  
3   pinMode(LED_BUILTIN, OUTPUT);  
4 }  
5  
6 void loop() {  
7   // Turn the built-in LED off  
8   digitalWrite(LED_BUILTIN, HIGH);  
9   delay(1000);  
10  // Turn the built-in LED on  
11  digitalWrite(LED_BUILTIN, LOW);  
12  delay(1000);  
13 }
```

For the Portenta C33, the `LED_BUILTIN` macro represents the **green LED** of the built-in RGB LED of the board.

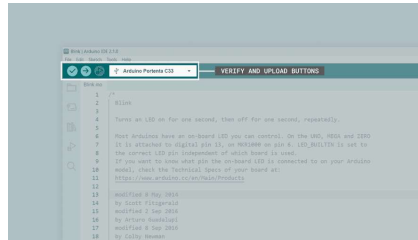


Built-in RGB LED of the Portenta C33 board

The built-in RGB LED on the Portenta C33 needs to be pulled to ground to make them light up. This means that a voltage level of `LOW` on each of their pins will turn the specific color of the LED on, and a voltage level of `HIGH` will turn them off.

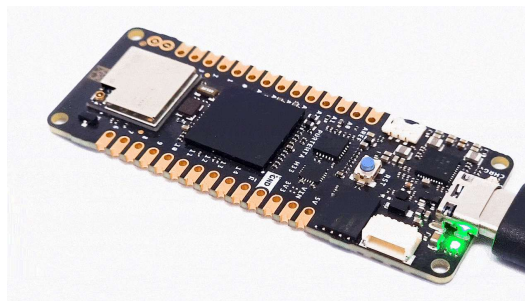
Furthermore, invoking the `pinMode(LED_BUILTIN, OUTPUT)` instruction pulls the built-in LED LOW, which means it turns it on.

To upload the code to the Portenta C33, click the **Verify** button to compile the sketch and check for errors; then click the **Upload** button to program



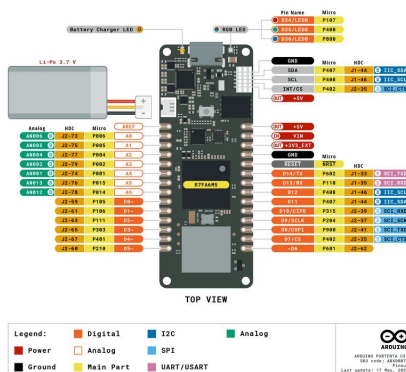
Uploading a sketch to the Portenta C33
in the Arduino IDE

You should see now the green LED of the built-in RGB LED turn on for one second, then off for one second, repeatedly.



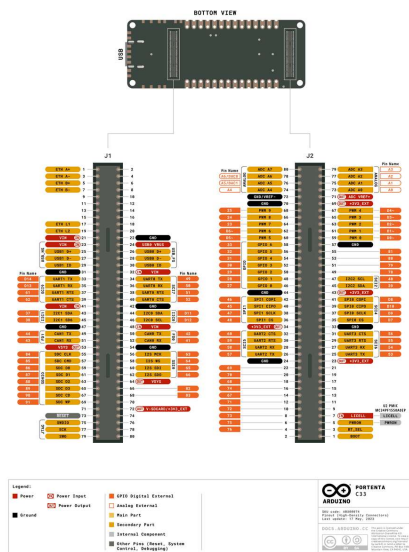
Pins

The Portenta C33 has been designed with two types of connectors: the MKR-styled connectors, intended for prototyping and debugging, and the High-Density connectors, intended to be used across Arduino Pro boards or custom boards. The Portenta C33 MKR-styled connectors pinout is shown in the image below:



Portenta C33 MKR-styled connectors
pinout

The Portenta C33 High-Density connectors pinout is shown in the image below:



Portenta C33 High-Density connectors
pinout

Analogue Pins

The Portenta C33 has eight analogue input pins mapped as follows:

Arduino Pin Mapping	Microcontroller Pin
A0	P006
A1	P005
A2	P004
A3	P002
A4	P001
A5 / DAC1	P015
A6 / DAC0	P014
A7	P000

The eight analogue input pins can be used through the built-in functions of the Arduino programming language (`analogRead()` function).

Please refer to the [board pinout section](#) of the user manual to find the analogue pins on the board:

MKR-style connectors: pins A0, A1, A2, A3, A4, A5, and A6 are accessible through these connectors.

High-Density connectors: pins A0, A1, A2, A3, A4, A5, A6, and A7 are accessible through these connectors.

The example code shown below reads the voltage value from a potentiometer connected to `A0` and displays it on the Arduino IDE Serial Monitor:

```
1 // Define the potentiometer pin and
2 int potentiometerPin = A0;
3 int potentiometerValue = 0;
4
5 void setup() {
6   // Initialize Serial communicatio
7   Serial.begin(9600);
8 }
9
10 void loop() {
11   // Read the voltage value from th
12   potentiometerValue = analogRead(p
13
14   // Print the potentiometer voltag
15   Serial.print("- Potentiometer vol
16   Serial.println(potentiometerValue
17
18   // Wait for 1000 milliseconds
19   delay(1000);
20 }
```

Digital Pins

The Portenta C33 has 91 digital input/output pins, mapped as described in the following tables. **The first 14 digital pins are available through the MKR-styled connectors of the board:**

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
0	GPIO	P105
1	GPIO	P106
2	GPIO	P11
3	GPIO	P303
4	GPIO	P401
5	GPIO	P210
6	GPIO	P601
7 / CS0	GPIO/SPI	P402
8 / MOSI0	GPIO/SPI	P900
9 / SCK0	GPIO/SPI	P204
10 / MISO0	GPIO/SPI	P315
11 / SDA0	GPIO/I2C	P407

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
13 / RX0	GPIO/UART	P110
14 / TX0	GPIO/UART	P602

The following eight digital pins also have **analog functionalities**; they are available through the MKR-styled and High-Density connectors of the board:

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
15 / A0	GPIO/Analog	P006
16 / A1	GPIO/Analog	P005
17 / A2	GPIO/Analog	P004
18 / A3	GPIO/Analog	P002
19 / A4	GPIO/Analog	P001
20 / A5 / DAC1	GPIO/Analog	P015
21 / A6 / DAC0	GPIO/Analog	P014
22 / A7	GPIO/Analog	P000

The following four digital pins are **PWM-capable**; they are available through the High-Density connectors of the board:

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
23	GPIO	P605
24	GPIO	P608
25	GPIO	P311
26	GPIO	P600

The following seven pins are **Interrupt Request (IRQ) capable**; they are available through the High-Density connectors of the board:

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
27	GPIO/IRQ	P009
28	GPIO/IRQ	P409
29	GPIO/IRQ	P505
30	GPIO/IRQ	P706

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
32	GPIO/IRQ	P708
33	GPIO/IRQ	P802

The following three pins are the digital pins used to control the **built-in RGB LED** of the board:

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
34 / LEDR	GPIO	P107
35 / LEDG	GPIO	P400
36 / LEDB	GPIO	P800

The following four digital pins are **I2C-capable**; they are available through the High-Density connectors of the board:

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
37 / SDA1	GPIO/I2C	P511
38 / SCL1	GPIO/I2C	P512
39 / SDA2	GPIO/I2C	P302
40 / SCL2	GPIO/I2C	P301

The following four digital pins are **CAN-capable**; they are available through the High-Density connectors of the board:

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
41	GPIO/CAN RX	P202
42	GPIO/CAN TX	P203
43	GPIO/CAN1 RX	P610
44	GPIO/CAN1 TX	P609

The following four digital pins are **SPI-capable**; they are available through the High-Density connectors of the board:

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
45	GPIO/MISO1	P100
46	GPIO/MOSI1	P101

The following 14 digital pins are **UART-capable**; they are available through the High-Density connectors of the board:

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
49	GPIO/TX2	P613
50	GPIO/RX2	P614
51	GPIO/RTS2	P611
52	GPIO/CTS2	P404
53	GPIO/TX3	P506
54	GPIO/RX3	P304
55	GPIO/RTS3	P503
56	GPIO/CTS3	P502
57	GPIO/TX4	P805
58	GPIO/RX4	P513
59	GPIO/RTS4	P508
60	GPIO/CTS4	P500
61	GPIO/RTS0	P603
62	GPIO/CTS0	P604

The following four digital pins are **SSI-capable**; they are available through the High-Density connectors of the board:

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
63	GPIO/SSI CK	P112
64	GPIO/SSI WS	P113
65	GPIO/SSI SDI	P114
66	GPIO/SSI SDO	P115

The following 17 digital pins are **generic GPIO pins**; they are available through the High-Density connectors of the board:

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
67	GPIO	P908
68	GPIO	P403
69	GPIO	P901
70	GPIO	P612

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
72	GPIO	P313
73	GPIO	P314
74	GPIO	PA01
75	GPIO	PA08
76	GPIO	PA09
77	GPIO	PA10
78	GPIO	P507
79	GPIO	PB00
80	GPIO	P615
81	GPIO	P003
82	GPIO	P007
83	GPIO	P008

The last eight digital pins are **SD-card-capable**; they are available through the High-Density connectors of the board:

Arduino Pin Mapping	Pin Functionality	Microcontroller Pin
84	GPIO/SDHI CLK	P413
85	GPIO/SDHI CMD	P412
86	GPIO/SDHI D0	P411
87	GPIO/SDHI D1	P410
88	GPIO/SDHI D2	P206
89	GPIO/SDHI D3	P205
90	GPIO/SDHI CD	P415
91	GPIO/SDHI WP	P414

The basic functionality of the Portenta C33's digital pins can be implemented through the built-in functions of the Arduino programming language. The configuration of a digital pin is done in the `setup()` function with the **built-in function** `pinMode()`, as shown below:

```
1 // Pin configured as an input
2 pinMode(pin, INPUT);
3
4 // Pin configured as an output
5 pinMode(pin, OUTPUT);
6
7 // Pin configured as an input, internal pull-up resistor
8 pinMode(pin, INPUT_PULLUP);
```

The state of a digital pin, configured as an input, can be read using the **built-in function**

`digitalRead()` as shown below:

```
1 // Reads pin state, stores value in state
2 state = digitalRead(pin);
```

The state of a digital pin, configured as an output, can be changed using the **built-in function**

`digitalWrite()` as shown below:

```
1 // Set pin on
2 digitalWrite(pin, HIGH);
3
4 // Set pin off
5 digitalWrite(pin, LOW);
```

The example code shown below uses digital pin `3` to control an LED depending on the state of a button connected to digital pin `2`:

```
1 // Define button and LED pin
2 int buttonPin = 2;
3 int ledPin = 3;
4
5 // Variable to store the button state
6 int buttonState = 0;
7
8 void setup() {
9   // Configure button and LED pins
10  pinMode(buttonPin, INPUT_PULLUP);
11  pinMode(ledPin, OUTPUT);
12
13  // Initialize Serial communication
14  Serial.begin(9600);
15 }
16
17 void loop() {
18   // Read the state of the button
19   buttonState = digitalRead(buttonPin);
20
21   // If the button is pressed, turn on the LED
22   if (buttonState == LOW) {
23     digitalWrite(ledPin, HIGH);
24     Serial.println("- Button is pressed");
25   } else {
26     // If the button is not pressed, turn off the LED
27     digitalWrite(ledPin, LOW);
28     Serial.println("- Button is not pressed");
29   }
30 }
```

PWM Pins

The Portenta C33 has 10 digital pins with PWM functionality, mapped as follows:

Arduino Pin Mapping	Microcontroller Pin
0 or D0	P105
1 or D1	P106
2 or D2	P111
3 or D3	P303
4 or D4	P401
5 or D5	P210
6 or D6	P601
7 or D7	P402
8	P900
9	P204
23	P605
24	P608
25	P311

The 10 PWM pins can be used through the built-in (`analogWrite()` function) of the Arduino programming language.

Please refer to the [board pinout section](#) of the user manual to find them on the board.

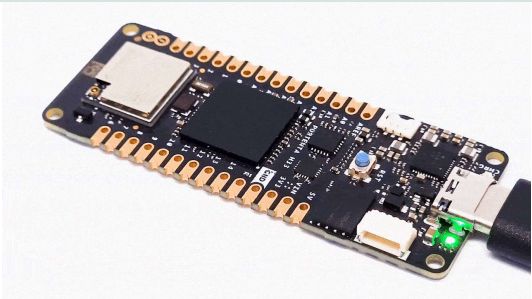
MKR-styled connectors: pins `0`, `1`, `2`, `3`, `4`, `5`, and `6` are accessible through these connectors.

High-Density connectors: pins `0`, `1`, `2`, `3`, `4`, `5`, `7`, `8`, `9`, `23`, `24`, `25`, and `26` are accessible through these connectors.

The example code shown below uses digital pin `9` PWM functionality to control the brightness of the board's built-in RGB LED (green LED) connected to it:

```
1 // Define the LED pin, brightness,
2 int brightness = 0;
3 int fadeAmount = 5;
4
5 void setup() {
6   // Configure the LED pin as an ou
7   pinMode(LEDG, OUTPUT);
8 }
9
10 void loop() {
11   // Set the brightness of the LED
12   analogWrite(LEDG, brightness);
13
14   // Update the brightness value
15   brightness += fadeAmount;
16
17   // Reverse the fade direction whe
18   if (brightness <= 0 || brightness
19       fadeAmount = -fadeAmount;
20 }
21
22 // Wait for 30 milliseconds
23 delay(30);
24 }
```

You should now see the built-in RGB LED's green LED fade in and then fade out repeatedly.

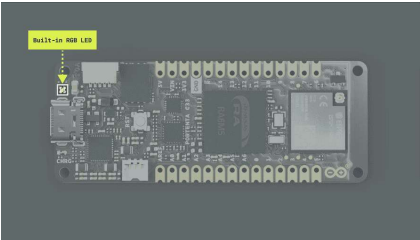


Actuators

This user manual section covers the Portenta C33 built-in actuators, showing their main hardware and software characteristics.

RGB LED

The Portenta C33 features a built-in RGB LED that can be used as a visual feedback indicator for the user.



Built-in RGB LED of the Portenta C33 board

The built-in RGB LED can be accessed through the following macro definitions:

Built-in LED	Macro Definition
Red LED	LEDR
Blue LED	LEDB
Green LED	LEDG or LED_BUILTIN

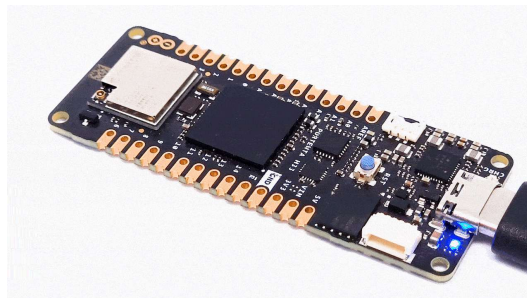
The built-in RGB LED on the Portenta C33 must be pulled to ground to make it light up. This means that a voltage level of `LOW` on each of their pins will turn the specific color of the LED on, and a voltage level of `HIGH` will turn them off. Furthermore, invoking the `pinMode(LED_BUILTIN, OUTPUT)` instruction pulls the built-in LED LOW, which means it turns it on.



The following example code blinks each of the LEDs at an interval of 500 ms:

```
1 void setup() {  
2   // Initialize LEDR, LEDG and LEDB  
3   pinMode(LEDR, OUTPUT);  
4   pinMode(LEDG, OUTPUT);  
5   pinMode(LEDB, OUTPUT);  
6 }  
7  
8 void loop() {  
9   // Turn on the built-in red LED a  
10  digitalWrite(LEDR, LOW);  
11  digitalWrite(LEDG, HIGH);  
12  digitalWrite(LEDB, HIGH);  
13  delay(500);  
14  // Turn on the built-in green LED  
15  digitalWrite(LEDR, HIGH);  
16  digitalWrite(LEDG, LOW);  
17  digitalWrite(LEDB, HIGH);  
18  delay(500);  
19  // Turn on the built-in blue LED  
20  digitalWrite(LEDR, HIGH);  
21  digitalWrite(LEDG, HIGH);  
22  digitalWrite(LEDB, LOW);  
23  delay(500);  
24 }
```

You should now see the built-in RGB LED blinking each of its LEDs repeatedly.



Communication

This section of the user manual covers the different communication protocols that are supported by the Portenta C33 board, including the Serial Peripheral Interface (SPI), Inter-Integrated Circuit (I2C), Universal Asynchronous Receiver-Transmitter (UART), and Wi-Fi®; JTAG interface and communication via the onboard ESLOV connector is also explained in this section.

MKR-styled connectors and the High-Density connectors, making connecting and communicating with different components, peripherals, and sensors easy.

SPI

The Portenta C33 supports SPI communication via two dedicated ports named `SPI0` and `SPI1`, which allows data transmission between the board and other SPI-compatible devices. The pins used in the Portenta C33 for the SPI communication protocol are the following:

Arduino Pin Mapping	Microcontroller Pin
<code>8</code> or <code>MOSI</code>	<code>P900</code>
<code>10</code> or <code>MISO</code>	<code>P315</code>
<code>9</code> or <code>SCK</code>	<code>P204</code>
<code>7</code> or <code>CS / SS</code>	<code>P402</code>
<code>46</code> or <code>PIN_SPI1_MOSI</code>	<code>P101</code>
<code>45</code> or <code>PIN_SPI1_MISO</code>	<code>P100</code>
<code>47</code> or <code>PIN_SPI1_SCK</code>	<code>P102</code>
<code>48</code> or <code>PIN_SPI1_CS</code>	<code>P103</code>

Please refer to the [board pinout section](#) of the user manual to find them on the board. Include the [SPI library](#) at the top of your sketch to use the SPI communication protocol. The SPI library provides functions for SPI communication:

```
1 #include <SPI.h>
```

In the `setup()` function, initialize the SPI library, define and configure the chip select (`CS`) pin:

```
1 void setup() {  
2   // Set the chip select pin as out  
3   pinMode(SS, OUTPUT);  
4  
5   // Pull the CS pin HIGH to unsele  
6   digitalWrite(SS, HIGH);  
7  
8   // Initialize the SPI communicati  
9   SPI.begin();
```

To transmit data to an SPI-compatible device, you can use the following commands:

```
1 // Replace with the target device's
2 byte address = 0x00;
3
4 // Replace with the value to send
5 byte value = 0xFF;
6
7 // Pull the CS pin LOW to select th
8 digitalWrite(SS, LOW);
9
10 // Send the address
11 SPI.transfer(address);
12
13 // Send the value
14 SPI.transfer(value);
15
16 // Pull the CS pin HIGH to unselect
17 digitalWrite(SS, HIGH);
```

I2C

The Portenta C33 supports I2C communication, which allows data transmission between the board and other I2C-compatible devices. There are three available I2C ports in the Portenta C33: I2C0, I2C1, and I2C2. The pins used in those interfaces for the I2C communication protocol are the following:

Arduino Pin Mapping	Microcontroller Pin
11 / SDA0	P407
12 / SCL0	P408
37 / SDA1	P511
38 / SCL1	P512
39 / SDA2	P302
40 / SCL2	P301

To locate these pins on the board, please refer to the [board pinout section](#) of the user manual. The SDA0 and SCL0 pins are also available through the onboard ESLOV connector of the Portenta C33.

System integrators can use the Portenta C33's High-Density connectors to expand the signals of the board to a custom-designed daughter board or carrier. Below is the I2C pins mapping on the board's High-Density connectors and shared resources:



HD Connector	Interface	Pins	Status	Shared Peripherals
J1	I2C1	43-45	Free	-
J2	I2C0	44-46	Free	-
J2	I2C2	45-47	Free	-

The status column indicates the current status of the pins. "Free" means the pins are not in use by another resource or peripheral of the board and are available for usage, while "Shared" means the pins are used by one or several resources or peripherals of the board.

To use I2C communication, include the `Wire` library at the top of your sketch. The `Wire` library provides functions for I2C communication:

```
1 #include <Wire.h>
```

In the `setup()` function, initialize the I2C library:

```
1 // Initialize the I2C communication
2 Wire.begin();
```

To transmit data to an I2C-compatible device, you can use the following commands:

```

1 // Replace with the target device's
2 byte deviceAddress = 0x1;
3
4 // Replace with the appropriate ins
5 byte instruction = 0x00;
6
7 // Replace with the value to send
8 byte value = 0xFF;
9
10 // Begin transmission to the target
11 Wire.beginTransmission(deviceAddress);
12
13 // Send the instruction byte
14 Wire.write(instruction);
15
16 // Send the value
17 Wire.write(value);
18
19 // End transmission
20 Wire.endTransmission();

```

To read data from an I2C-compatible device, you can use the `requestFrom()` function to request data from the device and the `read()` function to read the received bytes:

```

1 // The target device's I2C address
2 byte deviceAddress = 0x1;
3
4 // The number of bytes to read
5 int numBytes = 2;
6
7 // Request data from the target dev
8 Wire.requestFrom(deviceAddress, num
9
10 // Read while there is data availab
11 while (Wire.available()) {
12     byte data = Wire.read();
13 }

```

I2S

The Portenta C33 supports I2S (Inter-IC Sound), an interface primarily for transmitting high-quality audio data between digital devices. Unlike UART or SPI interfaces, which are more commonly known and used, the I2S interface is specifically designed for audio applications, making it an essential interface in modern audio electronics.

without losing quality. It operates using three main signals: `Serial Data` (`SD`), `Word Select` (`WS`), and `Serial Clock` (`SCK`). `Serial Data` can be input (`SDI`) or output (`SDO`). These signals work together to synchronize the transmission of audio samples between devices such as digital-to-analog converters (DACs), analog-to-digital converters (ADCs), and microcontrollers.

The pins used in the Portenta C33 for the I2S interface are the following:

Arduino Pin Mapping	Microcontroller Pin	I2S Interface
63	P112	CK
64	P113	WS
65	P114	SDI
66	P115	SDO

The I2S interface pins are accessible only through the Portenta C33's High-Density connectors (J1). You can access the pins using an Arduino Pro carrier board like the [Portenta Breakout](#), the [Portenta Hat Carrier](#), or the [Portenta Mid Carrier](#).

The `Arduino Renesas Core` has a built-in library that lets you use the I2S interface, the `I2S` library, right out of the box. Let's walk through an example sketch demonstrating some of the library's capabilities.

The example sketch below showcases configuring the I2S interface and playing a generated sine wave with the Portenta C33 board.



```

1  /**
2   I2S Sine Wave
3   Name: portenta_c33_i2s_sine_wave
4   Purpose: This sketch demonstrates
5   generating and playing a sine wave
6
7   @author Arduino Product Experier
8   @version 1.0 25/06/24
9  */
10
11 // Include the necessary library
12 #include "I2S.h"
13
14 // Define the sampling rate in Hz
15 #define SAMPLE_RATE 44100
16
17 // Constant used in the sine wave
18 #define TAU 6.2831853071795864769
19
20 // Buffer to hold the generated samples
21 uint16_t samplebuf[4096];
22
23 void setup() {
24   // Initialize serial communication
25   Serial.begin(115200);
26   for (auto startNow = millis() + 2500;
27        startNow < millis(); ) {
28     // Initialize the I2S interface
29     I2S.begin(I2S_OUTPUT, SAMPLE_RATE, 4096, 4);
30     // Generate a sine wave
31     for (int i = 0; i < 4096; i++) {
32       // Calculate the sine wave value
33       float value = sin((float)i * TAU / (float)SAMPLE_RATE);
34       // Convert the value to a 16-bit integer
35       int16_t sample = (int16_t)(value * 32768);
36       // Store the sample in the buffer
37       samplebuf[i] = sample;
38     }
39     // Print the buffer address
40     Serial.print("Buffer address: ");
41     Serial.println((void *)samplebuf);
42   }
43 }

```

Let's analyze the example sketch. First, the necessary configurations are made in the setup function. The `I2S.h` library is included to provide the necessary functions for I2S communication. Constants for the sampling rate (`SAMPLE_RATE`) and the value of `TAU` are defined for use in the sine wave calculations.

The setup function initializes serial communication for debugging purposes and waits up to 2.5 seconds for a connection. It then configures the I2S interface in output mode with a sampling rate of 44100 Hz, a buffer size of 4096 samples, and a queue depth of four. If the initialization fails, it prints an error message and enters an infinite loop. The function also generates a sine wave and stores it in the `samplebuf` buffer. The sine wave is generated for a 440 Hz tone (the standard `A` note), and the same sample is assigned to both the left and right audio channels.

In the loop function, a buffer is continuously retrieved to write data using `I2S.dequeue()` function. The pre-generated sine wave samples

then written to the I2S interface using

```
I2S.write()
```

The sine wave generation involves calculating the samples for a 440 Hz tone and storing them for both the left and right audio channels. The samples are continuously written to the I2S interface to be played as audio output. You should hear a continuous 440 Hz tone when this sketch is uploaded and running on the Portenta C33. You can modify the frequency, sampling rate, and buffer size to experiment with different audio signals and configurations.

You can download the example sketch [here](#).

UART

The Portenta C33 supports UART communication. The pins used in the Portenta C33 for the UART communication protocol are the following:

Arduino Pin Mapping	Microcontroller Pin
13 / RX0	P110
14 / TX0	P602
49	P613
50	P614
53	P506
54	P304
57	P805
58	P513
92	P603
93	P604

i

Please refer to the board pinout section of the user manual to find them on the board. The built-in `Serial` library functions can use the UART pins.

The `Arduino Renesas Core` has a built-in library that lets you use the UART communication, the `Serial` library, right out of the box. Let's walk through an example sketch demonstrating some of the module's capabilities.

The example sketch below showcases how to configure the UART interface, read incoming data,

which are common tasks for serial communication.

```
1  /**
2   * UART Communication
3   * Name: UARTCommunication.ino
4   * Purpose: This sketch demonstrates
5   *
6   * @author Arduino Product Experience
7   * @version 1.0 03/06/24
8   */
9
10 // Include the necessary libraries
11 #include <Arduino.h>
12
13 // Variable for storing incoming data
14 String incoming = "";
15
16 void setup() {
17     // Initialize serial communication
18     Serial.begin(115200);
19     for (auto startNow = millis() + 1000; ; startNow += 1000) {
20         // Print a message to the Serial Monitor
21         Serial.println("- UART communication initialized");
22     }
23 }
24
25 void loop() {
26     // Check for available data and read it
27     while (Serial.available()) {
28         // Allow data buffering and read it
29         char c = Serial.read();
30         incoming += c;
31     }
32     // Process the received data
33     processData();
34 }
```

Let's analyze the example sketch. First, the necessary configurations are made:

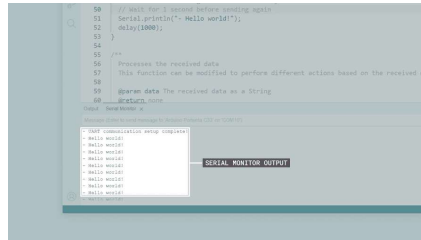
The UART communication is initialized at a baud rate of 115200.

A loop continuously checks for available data and reads individual characters, storing them in a `String` variable.

A newline character indicates the end of a message, triggering the processing function.

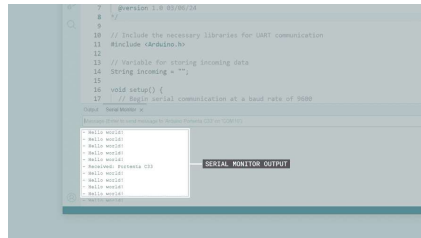
The `processData()` function is called to process the received data. This example simply prints the data to the Arduino IDE's Serial Monitor. You can modify this function to perform different actions based on the received data. Finally, the example sketch shows how to send data using the `Serial.println()` function, which transmits the string `Hello world!` every second.

You should see the following output in the Arduino IDE's Serial Monitor:



Example sketch output in the Arduino IDE's Serial Monitor

You can also send information to the Portenta C33 using the Arduino IDE's Serial Monitor. In the **Message** box of the IDE's Serial Monitor, write a message (for example, **Portenta C33**) and press Enter; you should see the following output in the Arduino IDE's Serial Monitor:

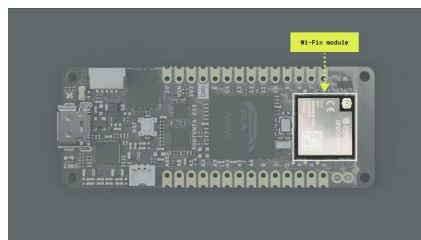


Example sketch output in the Arduino IDE's Serial Monitor

You can download the example sketch [here](#).

Wi-Fi®

The Portenta C33 board features an onboard Wi-Fi® module, the ESP32-C3-MINI-1U module from Espressif Systems®. The ESP32-C3-MINI-1U is a low-power, highly integrated Wi-Fi® and Bluetooth® System-on-Chip (SoC) solution designed for many IoT applications.



Onboard Wi-Fi® module of the Portenta C33 board

Some of the key capabilities of the ESP32-C3-

Wi-Fi® and Bluetooth® connectivity: The module supports 2.4 GHz Wi-Fi® (802.11 b/g/n) and Bluetooth® 5.0 connectivity.

`ArduinoBLE` library support is limited to Bluetooth® 4.0 at this time.

CPU and memory: It contains a 32-bit RISC-V single-core processor with a clock speed of up to 160 MHz. The chip also has 400 KB of SRAM and 384 KB of ROM.

Security features: It supports various security features, including secure boot, flash encryption, and cryptographic hardware acceleration.

Low-power operation: It supports multiple power modes for different low-power applications, making it suitable for battery-powered devices.

The Arduino Renesas Core has a built-in library that lets you use the onboard Wi-Fi® module, the `WiFiC3` library, right out of the box. Let's walk through an example code demonstrating some of the module's capabilities.

 Always connect your board's Wi-Fi® module with its W.FL antenna (included with your board) to avoid damaging it.

The code below showcases how to connect to a Wi-Fi® network, check Wi-Fi® status, connect to a server, send HTTP requests, and receive and print HTTP responses, which are common tasks for an IoT device.

```

1  /**
2   Web Client (Wi-Fi version)
3   Name: WiFiWebClient.ino
4   Purpose: This sketch connects to a web server
5
6   @author Arduino Team
7   @version 2.0 31/05/12
8  */
9
10 // Include the necessary libraries
11 #include "WiFiC3.h"
12 #include "WiFiClient.h"
13 #include "IPAddress.h"
14 #include "arduino_secrets.h"
15
16 // Define the credentials of the network
17 char ssid[] = SECRET_SSID; // Name of your network
18 char pass[] = SECRET_PASS; // Password of your network
19
20 // Define a variable for storing the status of the connection
21 int status = WL_IDLE_STATUS;
22
23 // Define the server to which we want to connect
24 // This can be an IP address or a domain name
25 char server[] = "www.google.com"
26
27 // Initialize the Wi-Fi client object
28 // This will be used to interact with the server

```

First, the necessary libraries are included:

The `WiFiC3.h` and `WiFiClient.h` are included at the start, those libraries contains the functionalities required to communicate via Wi-Fi®.

The SSID and password for the Wi-Fi® network are defined.

Then, the server is defined (`www.google.com` in this case) and the Wi-Fi® client object is created to manage the connection to the server.

Next, in the `setup()` function:

The serial port is initialized at a baud rate of 115200.

The sketch checks for Wi-Fi® module availability and its firmware version, then attempts to connect to the Wi-Fi® network with the defined SSID and password. If the connection is successful, it prints the Wi-Fi® status and attempts to connect to the defined server. If the server connection is successful, it sends a GET request.

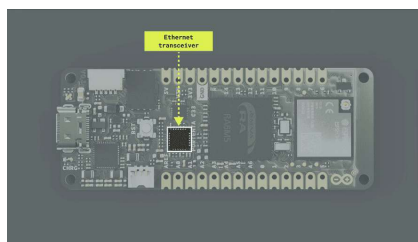
The `read_reponse()` function reads data from the client while there's data available. It wraps the data to 80 columns and prints it to the IDE's Serial Monitor.

The `read_response()` function is called to read any available data. If the server is disconnected, it disconnects the client and enters an infinite loop, halting the sketch.

The `printWifiStatus()` function prints the connected network SSID, the IP address of the board, and the signal strength (RSSI) to the IDE's Serial Monitor.

Ethernet

The Portenta C33 board features an Ethernet physical layer (PHY) transceiver, the LAN8742AI from Microchip®. The LAN8742AI is a low-power 10BASE-T/100BASE-TX physical layer (PHY) transceiver. This device is responsible for transmitting and receiving data over an Ethernet connection. The transceiver complies with the IEEE 802.3 and 802.3u standards and supports communication with an Ethernet MAC through a standard RMII interface.



Onboard Ethernet PHY transceiver of the Portenta C33 board

Some of the key capabilities of the LAN8742AI are the following:

Speed and duplex mode: It can operate at 10 Mbps (10BASE-T) or 100 Mbps (100BASE-TX). It also features auto-negotiation, which means it can automatically determine the best speed and duplex mode for communication.

HP Auto-MDIX: This feature allows you to use a straight-through or crossover Ethernet cable.

Wake on LAN (WoL): The device can be programmed to detect certain types of packets and trigger an interrupt.

Cable diagnostics: The transceiver can detect issues with the Ethernet cable and determine its location.

The Arduino Renesas Core has a built-in library that lets you use the onboard Ethernet PHY

library. Let's walk through an example code demonstrating some of the transceiver's capabilities. You can access your board's Ethernet pins through its High-Density connectors or an Arduino Pro carrier board like the [Portenta Breakout](#).

```

1  /**
2   Web Client (Ethernet version)
3   Name: WebClient.ino
4   Purpose: This sketch connects to a web server
5
6   @author Arduino Team
7   @version 4.0 01/06/18
8  */
9
10 // Include the Ethernet library
11 #include <EthernetC33.h>
12
13 // Define the server to which we want to connect
14 // This can be an IP address or a domain name
15 char server[] = "www.google.com"
16
17 // Set a static IP address to use if the DHCP fails
18 IPAddress ip(10, 130, 22, 84);
19
20 // Initialize the Ethernet client library
21 // This will be used to interact with the server
22 EthernetClient client;
23
24 void setup() {
25   // Begin serial communication
26   Serial.begin(115200);
27
28   // Wait for the serial port to open
29   while (!Serial) {
30     ; // Do nothing
31   }
32 }
33
34 void loop() {
35   // Check if the client has connected to the server
36   if (client.connected()) {
37     // Send a GET request to the server
38     client.print("GET / HTTP/1.1");
39     client.print("Host: www.google.com");
40     client.print("Connection: close");
41     client.print("\n");
42     // Wait for the server to send a response
43     while (client.available()) {
44       // Read the response from the server
45       char c = client.read();
46       // Print the response to the serial port
47       Serial.write(c);
48     }
49   }
50   // Wait for 10 seconds before connecting again
51   delay(10000);
52 }

```

First, the necessary libraries are included:

The `EthernetC33` library which contains the functionality required to communicate via Ethernet is included in the beginning.

Then, the server is defined, which is

`www.google.com` in this case:

The static IP address which will be used if the DHCP fails to assign an IP address is set.

The Ethernet client is initialized with the server's IP address and port.

Next, in the `setup()` function:

The serial port is initialized at a baud rate of

An Ethernet connection is attempted; if the DHCP configuration fails, the sketch tries to configure the connection using the static IP address defined before. The usage of a DNS server depends also on the success of the DHCP configuration.

If the DNS server is used, it connects to the server using the server name; otherwise, it uses the server IP address. If the connection is successful, it sends a `HTTP GET` request to the server. If not, it outputs an error message to the IDE's Serial Monitor.

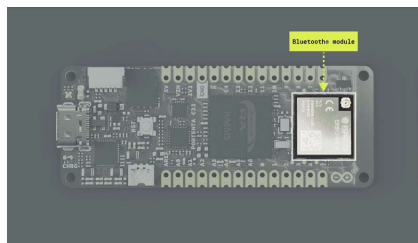
The `read_request()` function reads data from the client while there's data available. It wraps the data to 80 columns and prints it to the IDE's Serial Monitor.

Finally, in the `loop()` function:

The `read_request()` function is called to read any available data. If the server is disconnected, it disconnects the client and enters an infinite loop, halting the sketch.

Bluetooth®

The Portenta C33 board features an onboard Bluetooth® module, the ESP32-C3-MINI-1U module from Espressif Systems®. The ESP32-C3-MINI-1U is a low-power, highly integrated Wi-Fi® and Bluetooth® System-on-Chip (SoC) solution designed for many IoT applications.



Onboard Bluetooth® module of the Portenta C33 board



Always connect your board's Bluetooth® module with its W.FL antenna (included with your board) to avoid damaging it.

To enable Bluetooth® communication on the Portenta C33, you can use the `ArduinoBLE` library. Let's walk through an example code demonstrating some of its Bluetooth® module's capabilities. Here is an example of how to use the `ArduinoBLE` to create


```

1  /**
2   Portenta_C33_Bluetooth
3   Name: Portenta_C33_Bluetooth.ino
4   Purpose: Read voltage level from
5   then maps the voltage reading to
6
7   Then exposes the voltage percent
8
9   @author Arduino Team
10  @version 1.1 13/07/23
11  */
12
13  #include <ArduinoBLE.h>
14
15  // Define the voltage service and
16  BLEService voltageService("1101")
17  BLEUnsignedCharCharacteristic vol
18
19
20  const int analogPin = A0;
21
22  /**
23   Read voltage level from an analog
24   then maps the voltage reading to
25
26   @param none
27   @return the voltage level percent
28  */

```

The example code shown above creates a Bluetooth® Low Energy service and characteristic for transmitting a voltage value read by one of the analog pins of the Portenta C33 to a central device.

The code begins by importing all the necessary libraries and defining the Bluetooth® Low Energy service and characteristics.

In the `setup()` function, the code initializes the Portenta C33 board and sets up the Bluetooth® Low Energy service and characteristics. Then, it begins advertising the defined Bluetooth® Low Energy service.

A Bluetooth® Low Energy connection is constantly verified in the `loop()` function; when a central device connects to the Portenta C33, its built-in blue LED is turned into a solid blue. The code then enters into a loop that constantly reads the voltage level from an analog input and maps it to a percentage value between 0 and 100. The voltage level is printed to the Serial Monitor and transmitted to the central device over the defined Bluetooth® Low Energy characteristic.

JTAG

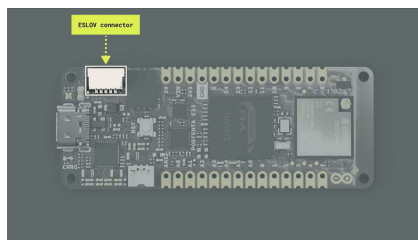
The Portenta C33 board features a JTAG/SWD debug port accessible through its High-Density connectors. The pins used for the JTAG/SWD debug port are the following:

Functionality	Microcontroller Pin
TMS/SWD	P108
TCK/SCK	P300
TD0/SWO	P109

You can use a [Portenta Breakout](#) with your Portenta C33 board to easily access your board's JTAG/SWD debug port through the individual pins or a dedicated MIPI 20-pin JTAG connector with trace capability.

ESLOV Connector

The Portenta C33 board features an onboard ESLOV connector to extend the I2C communication bus. This connector simplifies connecting various sensors, actuators, and other modules to the Portenta C33 without soldering or wiring; Nicla family boards can also be connected to the Portenta C33 through this connector.



Portenta C33 built-in ESLOV connector

The ESLOV connector is a small 5-pin connector with a 1.00 mm pitch. The mechanical details of the connector can be found in the [connector's datasheet](#).

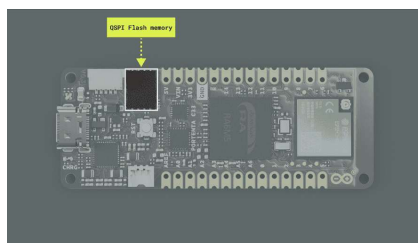
The pin layout of the ESLOV connector is the following:

- 1 VCC
- 2 INT
- 3 SCL
- 4 SDA

`VCC` pin works as a 5 V output if the board is connected to a USB-C® cable. The manufacturer part number of the ESLOV connector is SM05B-SRSS, and its matching receptacle manufacturer part number is SHR-05V-S-B.

External Memory

The Portenta C33 board features an onboard 16 MB QSPI Flash memory, the MX25L12833F from Macronix®. Having an onboard QSPI Flash memory enhances the capabilities of the Portenta C33, enabling you to develop and run more complex and data-intensive applications.



Onboard QSPI Flash memory of the Portenta C33 board

Some key advantages of having an onboard QSPI Flash memory are the following:

Storage capacity: The MX25L12833F QSPI Flash memory adds significant non-volatile storage to the board.

Extended functionality: The additional memory space allows more complex applications to be developed and run on your Portenta C33. This application includes data logging, image processing, audio processing, and executing complex algorithms.

Firmware updates: The MX25L12833F QSPI Flash memory can also store firmware or software updates for your Arduino board. This means you can update the firmware without requiring a complete reprogramming of the board.

The Arduino Renesas Core has built-in libraries and drivers that immediately let you use the onboard QSPI Flash memory. Let's walk through an example code demonstrating some of the onboard Flash memory capabilities; the main tasks performed in the sketch are initializing the Flash memory, writing to a file, and reading from

```

1  /**
2   QSPI Flash File System test
3   Name: QSPI_Flash_FileSystem_Test
4   Purpose: This sketch test the on
5   file system by writing and read
6
7   @author Arduino Team
8   @version 1.0 20/06/23
9  */
10
11 // Include necessary libraries and
12 #include "QSPIFlashBlockDevice.h"
13 #include "FATFileSystem.h"
14
15 // Define constants for file syste
16 #define QSPI_FS_NAME "qspi"
17 #define TEST_FILE "test.txt"
18
19 // Instantiate flash block device
20 QSPIFlashBlockDevice block_device;
21 FATFileSystem fs(QSPI_FS_NAME);
22
23 // Define full path to the test f:
24 std::string file_test_name = std:
25
26 void setup() {
27     // Initialize serial communicat
28     Serial.begin(9600);

```

Here's what each section of the example code does:

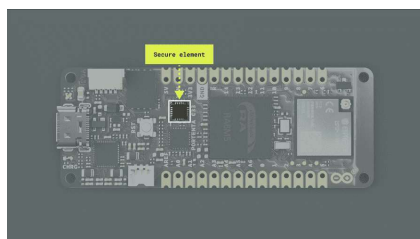
Initialize the onboard Flash memory: After setting up the Serial port, the sketch tries to mount the QSPI Flash file system. If the file system is not found, the code formats the memory to set up a new file system. If there's an error during this process, the code will halt and print an error message to the IDE's Serial Monitor.

Write to a file: Next, the sketch attempts to open a file named `test.txt` in `write` mode. If this process is successful, it writes the string `Hello from QSPI Flash!` to the file and then closes it. If it cannot open the file, it will print an error message to the IDE's Serial Monitor.

Read from a file: After writing to the file, the sketch attempts to open the same file, but this time in 'read' mode. If successful, it will read the content of the file one character at a time and print each character to the IDE's Serial Monitor. After reading, it closes the file. If it cannot open the file for reading, it will print an error message to the IDE's Serial Monitor.

Secure Element

The Portenta C33 features a secure element onboard, the SE050 from NXP®. A secure element is a component that can store sensitive data and run secure apps; it acts as a vault, protecting what is inside the secure element from attacks. A secure element simplifies the implementation of robust security mechanisms, reducing time to market without requiring specialized security expertise.



Onboard secure element of the
Portenta C33 board

The SE050 is a ready-to-use IoT secure element that provides a root of trust at the IC level and gives an IoT system state-of-the-art, edge-to-cloud security capability. It allows for securely storing and provisioning credentials and performing cryptographic operations for security-critical communication and control functions. The Arduino Renesas Core has a built-in library that lets you use the secure onboard element right out of the box, the `SE05X` library.

The Arduino sketch below can help you start with the SE050 and demonstrate its capabilities. The example code generates a new Elliptic Curve (EC) key pair (NIST Curve P-256), signs a SHA256 hash of a predefined input with the private key, and verifies the signature with the public key.



```

1  /**
2   SE05X Import and Verify
3   Name: import_public_key.ino
4   Purpose: This sketch generates
5   signs a SHA256 hash of a predefi
6   and verifies the signature with
7
8   @author Arduino HW/FW team, mod
9   @version 1.0 08/06/23
10 */
11
12 // Include the SE05X library
13 #include <SE05X.h>
14
15 // Define IDs for the private and
16 const int PRIVATE_KEY_ID = 999;
17 const int PUBLIC_KEY_ID = 899;
18
19 // Define the input buffer
20 const byte input[64] = {
21     0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F,
22     0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F,
23     0x20, 0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27, 0x28, 0x29, 0x2A, 0x2B, 0x2C, 0x2D, 0x2E, 0x2F,
24     0x30, 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37, 0x38, 0x39, 0x3A, 0x3B, 0x3C, 0x3D, 0x3E, 0x3F,
25 };
26
27 /**
28  Print a buffer in hexadecimal f

```

Here's what each section of the example code does:

Input buffer initialization: The sketch begins by initializing a 64-byte input buffer.

Buffer print function: A function named `printBufferHex()` is defined to print the contents of a buffer in hexadecimal format to the IDE's Serial Monitor.

Secure element initialization: In the `setup()` function, communication with the secure element is initiated. If it fails, an error message is shown in the IDE's Serial Monitor, and the program is halted.

Private key generation: The secure element generates an Elliptic Curve (EC) NIST P-256 private key. The key ID is set to 999, and the generated key is returned in DER format.

Input hash and signature: The SHA256 hash of the input buffer is computed and printed, then signed using the generated private key.

Public key import: The public key derived from the generated private key is imported to the secure element with key ID 899.

Signature verification: The example code then verifies the signature using the

Arduino Cloud

The Portenta C33 is fully compatible with the Arduino Cloud IoT, which simplifies how professional applications are developed and tracked. By using the IoT Cloud, you can, for example, monitor sensor data, control your board and actuators connected to it remotely, and update your device's firmware over-the-air.

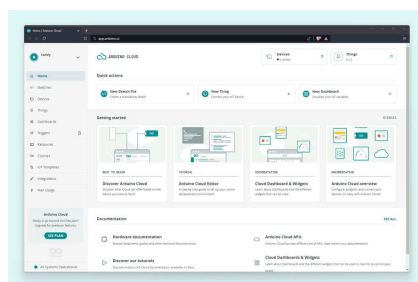
In case it is the first time you are using the Arduino Cloud:

To use the Arduino Cloud, you need an account. If you do not have an account, create one for free [here](#).

To use the Arduino Cloud Editor or Arduino Cloud, the Arduino Create Agent must be running on your computer. You can install the Arduino Create Agent [here](#).

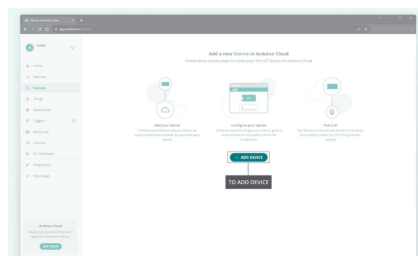
Let's walk through a step-by-step demonstration of how to use your Portenta C33 board with the Arduino Cloud.

Log in to your Arduino Cloud account; you should see the following:

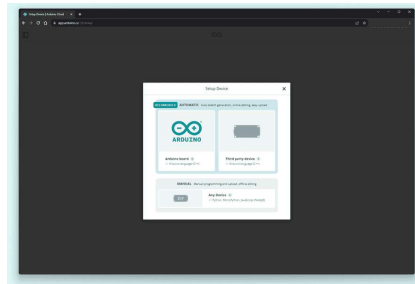


Arduino Cloud initial page

First, provision your Portenta C33 board on your Arduino Cloud space. To do this, navigate to **Devices** and then click on the **ADD DEVICE** button:

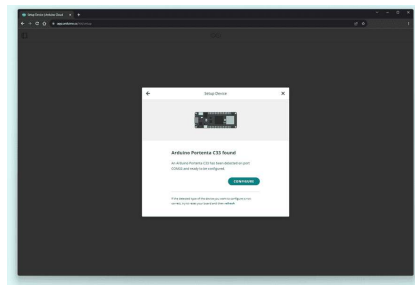


The **Setup Device** pop-up window will appear. Navigate into **AUTOMATIC** and select the **Arduino board** option:



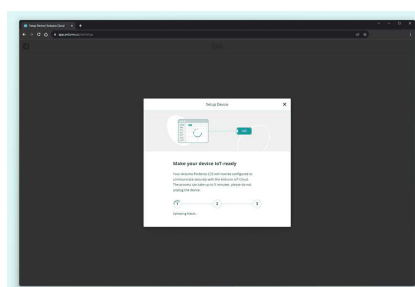
Arduino Cloud Setup Device pop-up window

After a while, your Portenta C33 board should be discovered by the Arduino Cloud, as shown below:

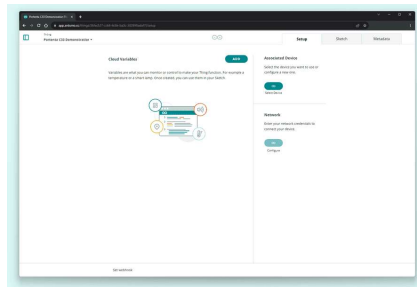


Arduino Cloud Setup Device pop-up window

Click the **CONFIGURE** button, give your board a name, and select the type of network connection. In this example, we will use a Wi-Fi® connection; you can also use an Ethernet connection with a **Portenta Max Carrier**, a **Portenta Breakout**, a **Portenta Vision Shield** or a custom-made board with an Ethernet connector. Your Portenta C33 board will be configured to securely communicate with the Arduino Cloud. This process can take a while.



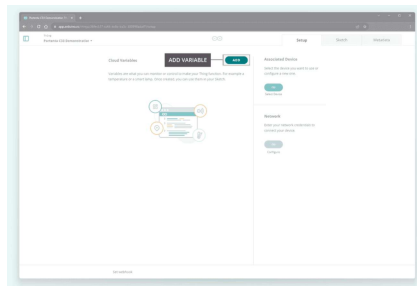
Once the Portenta C33 has been configured, let's create a "Thing" to test the connection between your board and the Arduino Cloud. Navigate into **Things** and select the **CREATE THING** button; give your thing a name.



Arduino Cloud "Thing" setup

Navigate into **Associate Device** and click the **Select Device** button. Select your Portenta C33 board and associate it with your "Thing." Then, navigate into **Network** and click the **Configure** button; enter your network credentials.

The project is now ready to add some variables to your "Thing"; navigate into **Cloud Variables** and click the **ADD** button to add variable.



Add variable button

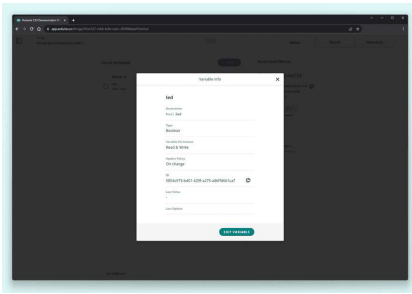
Add one variable with the following characteristics:

Name: led

Variable type: Boolean

Variable permission: Read & Write

Variable update policy: On change



Arduino Cloud "Thing" variable setup

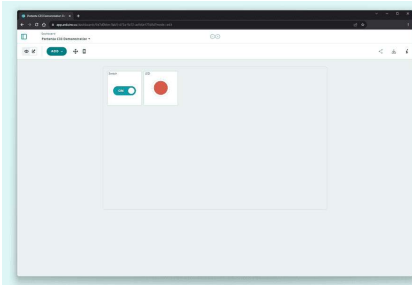
You should see the `led` variable in the **Cloud Variables** section. Navigate into **Dashboards** and select the **CREATE DASHBOARD** button; create a new dashboard and give your dashboard a name.

Add the following widgets to your dashboard:

Switch: name the widget `Switch` and link it to the `led` variable you created before.

LED: name the widget `led` and link it to the `led` variable you created before.

Your dashboard should look like the following:



Arduino Cloud Dashboard setup

Go back to your **Things** and open the "Thing" you created. In the "Thing" setup page, navigate into **Sketch**, where you should see the online editor.

In the generated sketch, define `LED_BUILTIN` pin as an output in the `setup()` function:

```
1 void setup() {
2   // Initialize serial and wait for
3   Serial.begin(9600);
4   // This delay gives the chance to
5   delay(1500);
6
7   // LED_BUILTIN macro access the c
8   pinMode(LED_BUILTIN, OUTPUT);
9
10  // Defined in thingProperties.h
11  initProperties();
12
13  // Connect to Arduino Cloud
14  ArduinoCloud.begin(ArduinoIoTPref
15
16  /*
17   The following function allows
18   related to the state of network
19   the higher number the more gra
20   The default is 0 (only errors)
21   Maximum is 4
22  */
23  setDebugMessageLevel(2);
24  ArduinoCloud.printDebugInfo();
25 }
```

In the `onLedChange()` function, which was generated automatically by the Arduino Cloud when the variable `led` was created, you must associate the onboard green LED state with the `led` variable:

```
1 /*
2   Since Led is READ_WRITE variable,
3   executed every time a new value is
4  */
5 void onLedChange() {
6   digitalWrite(LED_BUILTIN, !led);
7 }
```

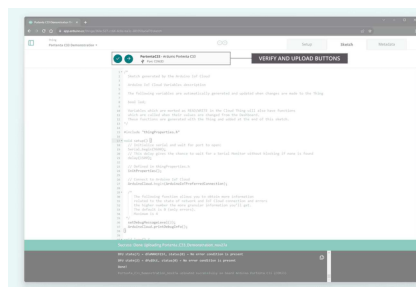
The complete example code can be found below:

```

1  /*
2   Sketch generated by the Arduino
3
4   Arduino Cloud Variables description
5
6   The following variables are automatically
7
8   bool led;
9
10  Variables which are marked as Read-Only
11  which are called when their value changes
12  These functions are generated with the Arduino Cloud
13  */
14
15  #include "thingProperties.h"
16
17  void setup() {
18    // Initialize serial and wait for port
19    Serial.begin(9600);
20    // This delay gives the chance to setup the
21    delay(1500);
22
23    // Defined in thingProperties.h
24    initProperties();
25
26    // Connect to Arduino Cloud
27    ArduinoCloud.begin(ArduinoIoTPreferredPlatform);
28

```

To upload the code to the Portenta C33 from the online editor, click the green **Verify** button to compile the sketch and check for errors, then click the green **Upload** button to program the board with the sketch.



Uploading a sketch to the Portenta C33
in the Arduino Cloud

Navigate into **Dashboards** again, your board should connect to the Wi-Fi® network you defined before (you can follow the connection process with the online editor integrated Serial Monitor). Your board's green LED should light on or off when the position of the switch changes.

If you encounter any issues or have questions while working with the Portenta C33, we provide various support resources to help you find answers and solutions.

Help Center

Explore our Help Center, which offers a comprehensive collection of articles and guides for the Portenta C33. The Arduino Help Center is designed to provide in-depth technical assistance and help you make the most of your device.

[Portenta C33 help center page](#)

Forum

Join our community forum to connect with other Portenta C33 users, share your experiences, and ask questions. The forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Portenta C33.

[Portenta C33 category in the Arduino Forum](#)

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Portenta C33.

[Contact us page](#)

Suggest changes

The content on docs.arduino.cc is facilitated through a public [GitHub repository](#). If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino](#)
[Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

